

宋林轲

unik-lif.github.io | Unik-lif

+86-18511165662 | songlinke@iie.ac.cn | 中国, 北京, 100190

个人介绍

中国科学院大学 / 信息工程研究所计算机系统方向博士研究生。工作涉及硬件 ISA、固件、Linux 内核内部机制及用户态——跨越系统全栈，项目独立完成。代表性工作：NestedSGX（在机密虚拟机内构建嵌套 Enclave；NDSS'25）、独立设计并实现的机密容器系统（定制 ISA + 内核关键路径修改 + 固件级安全执行；正在向 BOOM RISC-V 核与 FPGA 平台移植）、LLM KV Cache 侧信道攻击（负责 GPU 噪声缓解、攻击方案设计与实现；TPR 99%，TIFS'25）。研究方向：OS 级隔离、虚拟化、沙盒、TEE、可扩展机密计算，以及 LLM Agent 基础设施（容器沙盒、KV Cache 安全、隐私保护推理）。期望获得 Agent 系统、Agent 容器/沙盒管理、KV Cache 基础设施方向的实习机会。

教育背景

中国科学院大学

中国, 北京

计算机系统结构, 博士研究生

2024 年 9 月 - 至今

信息工程研究所 | 导师: 宋威教授, 王文浩教授

网络空间安全, 硕士

2022 年 9 月 - 2024 年 7 月

信息工程研究所 | 导师: 王文浩教授 | GPA: 3.83/4.00

网络空间安全, 学士

2018 年 9 月 - 2022 年 7 月

导师: 王文浩教授, 林东岱教授 | GPA: 3.70 (6/20)

研究项目

LLM 侧信道攻击: KV Cache 时序泄露

工具: Python, SGLang, GPT-Cache, LLaMAFactory

[代码](#) [论文](#) [demo](#) [SGLang #1504](#)

问题: LLM 推理服务中的 KV Cache 复用产生时序侧信道: 攻击者作为普通用户发送精心构造的请求时, 若其输入与受害者在 GPU 显存中缓存的 prompt 存在重叠, 即可观测到 TTFT (首 token 延迟) 缩短, 从而泄露受害者发送的内容。在 SGLang (前缀共享) 和 GPT-Cache (语义相似度匹配) 中均发现可利用。

方案: 在 SGLang 中, 使用 Llama 3.1 8B Instruct / 70B GPTQ INT4 时, 仅多共享一个 token 即可产生可测量的延迟下降。设计了逐 token 恢复受害者 prompt 的方法——逐位猜测下一个 token, 利用时序信号验证。单 token 时序差异极小且易被 GPU 电压/频率波动淹没, 为此设计了拮抗电压与频率干扰的时序测量方法, TPR 达到 99%。

GPT-Cache 攻击: 发现了 GPT-Cache 中一种不同的攻击方式——表达接近但敏感信息一致的查询因语义相似度匹配触发 TTFT 加速, 从而可推测有限集合内的隐私信息。

防御: 提出更大粒度的 token 共享防御方案, 扩展攻击者猜测空间, 显著降低提取成功率。

影响: 最早向 SGLang 项目报告 KV Cache 隐私风险的两个独立团队之一 (另一团队来自字节跳动安全研究部, 同一周报告); 于 2024 年 10 月 19 日在 SGLang 双周会上分享发现, 获得团队关注与认可。

发表: 投稿至 USENIX Security'25、ACM CCS'25; 被 TIFS'25 接收 (CCF-A 期刊)。

NaCRE: RISC-V 原生机密容器

工具: C, OpenSBI, Linux Kernel, RunC, Qemu, Chipyard, BOOM

[代码](#) | [海报 \(CCF-NSS 2026\)](#)

问题: 现有机密容器往往复用并非为容器范式设计的硬件机制 (如 TEE), 丧失了容器作为特殊进程的原生性和轻量度。部分替代方案 (如 Arm CCA) 将容器视为 TEE 工作负载, 依赖连续大块内存且无法在容器间共享——业界缺乏专门为原生机密容器设计的硬件原语与抽象。

方案: 复用 RISC-V 原生的 PMP (物理内存保护) 机制, 通过 bitmap 和针对 MMU 的修改, 在不碎片化 Linux 原生内存分配的前提下, 实现对用户页表页、用户数据页及用户态页表项的保护——容器仍为普通进程, 而非微型 VM。

与现有方案对比: 不同于 Kata Containers 和 gVisor 依赖虚拟化或机密计算硬件 (对 RISC-V 而言过于沉重, 也不适用于其他架构的低负载场景), NaCRE 引入了专为容器设计的硬件抽象层。保留 Linux 原生内存分配并进行定向安全加固, 避免了对内核的侵入式修改。

- **安全机制**：容器生命周期防护集成于地址分配全流程，在不牺牲容器原生性和性能的前提下实现充分的安全加固。
- **性能**：计算密集型任务相较原生 Docker 几乎无性能损失；内存密集型任务性能损失控制在 2 倍以内，显著优于 microVM 方案。
- **硬件移植（进行中）**：将 NaCRE 从 QEMU 模型移植到真实硬件平台——**BOOM** 乱序 RISC-V 核（基于 Chipyard）与 **FPGA** 原型（FireSim 流程），并对物理内存布局（OpenSBI/Linux/Agent/metadata/PTP 区域）参数化以适配 FPGA DRAM 配置。
- **硬件设计与移植工作**：与 [Yifan Bai](#) 共同讨论并最终调整了硬件设计；Yifan Bai 基于其对 RISC-V H 扩展的理解实现了 Agent Mode 的编程模型。本人撰写 QEMU↔RTL 语义对齐设计，冻结硬件契约：自定义 CSR（nacc_state、nacc_sagent/nacc_eagent、twin_entry、trampoline）、acall/aret 自定义指令、PTW 页表遍历检查、每 PFN 2-bit metadata bitmap、tag 变更时的 TLB 撤销；定义（访问来源 × 特权级 × 状态 × PFN tag）→ allow/fault 访问矩阵与 T0-T8 裸机冒烟测试套件，在 QEMU 与 Verilator 间共同验证，明确区分架构语义与仿真器便利行为（如 A/D 回写、cpu_abort()）。
- **个人贡献**：独立完成全栈设计与开发——设计定制硬件 ISA 原语，修改 Linux 内核关键路径（页分配、RSS 计数、page fault 处理，保持现有 metadata 正确性），集成 OpenSBI 固件，定义 eCall 接口与协议。准备投稿 arXiv。
- **NestedSGX：机密虚拟机内嵌套 Enclave**

工具: *Rust, C, Python, Linux* 内核模块, *Qemu, AMD SEV-SNP* [🔗 代码](#) [📄 论文](#)

 - **问题**：机密虚拟机面临 Guest OS 中 TCB 过大的问题，存在可利用的攻击面。现有方案缺乏在机密虚拟机内部建立可信 Enclave、同时将 Guest OS 排除在 TCB 之外的机制。
 - **方案**：利用 AMD SEV-SNP 的 VMPL（虚拟机特权级）机制，在机密虚拟机内部引入轻量级 hypervisor，对 Guest OS 进行降权——即使 Guest 内核被完全攻破也无法访问 Enclave 内存，大幅缩小 TCB。
 - **兼容性**：在 Occlum 和 Intel SGX SDK 基础上构建可信 Enclave 运行时，保持与现有 Intel SGX 生态的兼容性，无需修改的 SGX 应用可直接在嵌套 Enclave 中运行。
 - **工程实现**：修改 Linux 底层内核驱动；使用 Rust 撰写机密虚拟机内 hypervisor（**7,000+** 行代码），处理 page fault、系统错误路径及跨特权级跳转（自定义 trampoline 机制）。
 - **认可**：获 **2 枚 Artifact Evaluation 徽章**；受星绽（Asterinas）社区邀请进行线上分享。投稿至 **ASPLOS'24**、**ACM CCS'24**；被 **NDSS'25** 接收（CCF-A 会议）。

专利与论文

C= 会议, J= 期刊, P= 专利, S= 投稿中, T= 学位论文

- [J.1] The Early Bird Catches the Leak: Unveiling Timing Side Channels in LLM Serving Systems.
[Linke Song, Zixuan Pang], Wenhao Wang*, Zihao Wang, XiaoFeng Wang, Hongbo Chen, Wei Song, Yier Jin, Dan Meng, Rui Hou
TIFS' 25
- [C.1] The Road to Trust: Building Enclaves within Confidential VMs.
Wenhao Wang, **Linke Song** (first student author), Benshan Mei, Shuang Liu, Shijun Zhao, Shoumeng Yan*, XiaoFeng Wang, Dan Meng, Rui Hou
NDSS'25

技能

- **编程语言**: C, Rust, Python
- **工具**: Git, Docker, Qemu, OpenSBI, Linux Kernel
- **专业领域**: 可信执行环境, 操作系统, 虚拟化, 容器
- **熟悉架构**: x86, RISC-V
- **自学经历**: [course_notes](#) —2022 年起系统性重建 CS 基础